

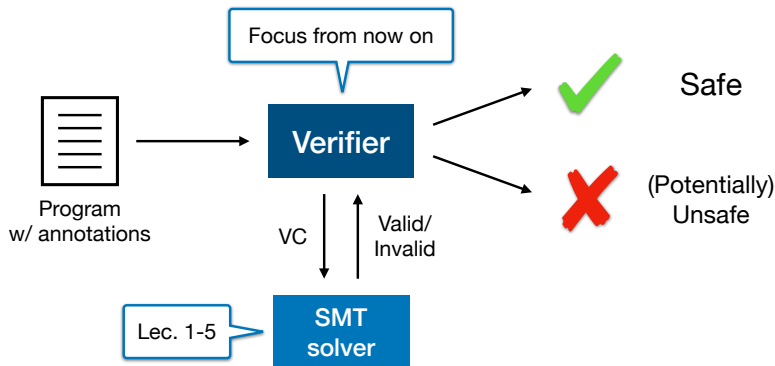
EC4219: Software Engineering

Lecture 6 — Program Verification (1) *Formal Specifications*

Sunbeom So
2025 Spring

Where We Are

- In Lectures 1–5, we explored the basic internals of SMT solvers.
- In the upcoming lectures, we will treat SMT solvers as black boxes and learn basic verification techniques.



Overview: Program Verification

We will learn methods for specifying and verifying program properties.

- **Specification:** precise statement of program properties in first-order logic (FOL). We focus on specifying two forms of properties.
 - ▶ **Partial correctness properties:** If a program halts, it produces a correct output that satisfies the specification.
 - ▶ **Total correctness properties:** A program is guaranteed to terminate and produce a correct output (partial correctness + termination).
- **Verification**
 - ▶ **Inductive assertion method** for proving partial correctness
 - ▶ **Ranking function method** for proving total correctness

Specification

- A specification consists of program annotations expressed as FOL formulas.
- An annotation F at location L expresses an **invariant** asserting that F is *true* whenever L is reached during the program execution.
- We explore three kinds of annotations.
 - 1 Function specification
 - 2 Loop invariant
 - 3 Assertion

Function Specification

Formulas whose free variables include the formal input and output parameters (and possibly global variables).

- **Precondition:** Specification that should be true **before entering** the function. In other words, it specifies under which inputs the function is expected to work.
- **Postcondition:** Specification that should be true **after exiting** the function. It may define the relationship between the input parameters and output parameters.

Example 1: Linear Search

```
1  @pre:
2  @post:
3  bool LinearSearch (int  $a[]$ , int  $l$ , int  $u$ , int  $e$ ) {
4      int  $i := l$ ;
5      while ( $i \leq u$ ) {
6          if ( $a[i] = e$ ) return true;
7           $i := i + 1$ ;
8      }
9      return false;
10 }
```

- @pre:
- @post:

Example 1: Linear Search

```
1  @pre:  $0 \leq l \wedge u < |a|$ 
2  @post:
3  bool LinearSearch (int  $a[]$ , int  $l$ , int  $u$ , int  $e$ ) {
4      int  $i := l$ ;
5      while ( $i \leq u$ ) {
6          if ( $a[i] = e$ ) return true;
7           $i := i + 1$ ;
8      }
9      return false;
10 }
```

- @pre: behaves correctly only when $l \geq 0$ and $u < |a|$.
- @post:

Example 1: Linear Search

```
1  @pre:  $0 \leq l \wedge u < |a|$ 
2  @post:  $rv \leftrightarrow \exists i. l \leq i \leq u \wedge a[i] = e$ 
3  bool LinearSearch (int  $a[]$ , int  $l$ , int  $u$ , int  $e$ ) {
4      int  $i := l$ ;
5      while ( $i \leq u$ ) {
6          if ( $a[i] = e$ ) return true;
7           $i := i + 1$ ;
8      }
9      return false;
10 }
```

- @pre: behaves correctly only when $l \geq 0$ and $u < |a|$.
- @post: returns *true* iff the array a contains e in the range $[l, u]$.

Example 2: Binary Search

```
1  @pre:
2  @post:  $rv \leftrightarrow \exists i. l \leq i \leq u \wedge a[i] = e$ 
3  bool BinarySearch (int  $a[]$ , int  $l$ , int  $u$ , int  $e$ ) {
4      if ( $l > u$ ) return false;
5      else {
6          int  $m := (l + u) \text{ div } 2$ ;
7          if ( $a[m] = e$ ) return true;
8          else if ( $a[m] < e$ ) return BinarySearch ( $a, m + 1, u, e$ )
9          else return BinarySearch ( $a, l, m - 1, e$ )
10     }
11 }
```

- @pre:
- @post: returns *true* iff the array *a* contains *e* in the range $[l, u]$.

Example 2: Binary Search

```
1  @pre:  $0 \leq l \wedge u < |a| \wedge \text{sorted}(a, l, u)$ 
2  @post:  $rv \leftrightarrow \exists i. l \leq i \leq u \wedge a[i] = e$ 
3  bool BinarySearch (int  $a[]$ , int  $l$ , int  $u$ , int  $e$ ) {
4      if ( $l > u$ ) return false;
5      else {
6          int  $m := (l + u) \text{ div } 2$ ;
7          if ( $a[m] = e$ ) return true;
8          else if ( $a[m] < e$ ) return BinarySearch ( $a, m + 1, u, e$ )
9          else return BinarySearch ( $a, l, m - 1, e$ )
10     }
11 }
```

- @pre: behaves correctly only when $l \geq 0$, $u < |a|$, and a is sorted.
- @post: returns *true* iff the array a contains e in the range $[l, u]$.

sorted is defined in the combined theory of integers and arrays ($T_{\mathbb{Z}} \cup T_A$).

$$\text{sorted}(a, l, u) \iff \forall i, j. l \leq i \leq j \leq u \rightarrow a[i] \leq a[j]$$

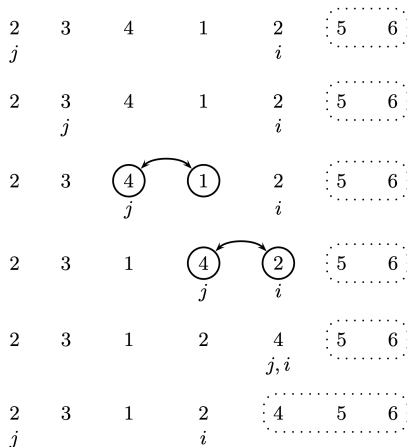
Example 3: Bubble Sort

```
1  @pre:
2  @post:
3  bool BubbleSort (int a0[]) {
4      int[] a := a0;
5      for (int i := |a| - 1; i > 0; i := i - 1) {
6          for (int j := 0; j < i; j := j + 1) {
7              if (a[j] > a[j + 1]) {
8                  int t := a[j];
9                  a[j] := a[j + 1];
10                 a[j + 1] := t;
11             }
12         }
13     }
14     return a;
15 }
```

- @pre:
- @post:

Example 3: Bubble Sort

BubbleSort works by repeatedly swapping adjacent elements if they are in the wrong order, effectively “bubbling” the largest element to the end of the unsorted region.



Example 3: Bubble Sort

```
1  @pre:  $\forall a. |a| \geq 0$ 
2  @post:  $\text{sorted}(rv, 0, |rv| - 1)$ 
3  bool BubbleSort (int  $a_0[]$ ) {
4      int[]  $a := a_0$ ;
5      for (int  $i := |a| - 1$ ;  $i > 0$ ;  $i := i - 1$ ) {
6          for (int  $j := 0$ ;  $j < i$ ;  $j := j + 1$ ) {
7              if ( $a[j] > a[j + 1]$ ) {
8                  int  $t := a[j]$ ;
9                   $a[j] := a[j + 1]$ ;
10                  $a[j + 1] := t$ ;
11             }
12         }
13     }
14     return  $a$ ;
15 }
```

- @pre: any array can be given as input.
- @post: the returned array is sorted.

Loop Invariant

```
1  @pre:  $n \geq 0$   
2  @post:  $j = n$   
3  void VerySimple (int  $n$ ) {  
4      int  $i := 0$ ;  
5      int  $j := 0$ ;  
6      while ( $i \neq n$ ) {  
7           $i := i + 1$ ;  
8           $j := j + 1$ ;  
9      }  
10     return;  
11 }
```

Q1. Does VerySimple satisfy the function specification?

Loop Invariant

```
1  @pre:  $n \geq 0$ 
2  @post:  $j = n$ 
3  void VerySimple (int  $n$ ) {
4      int  $i := 0$ ;
5      int  $j := 0$ ;
6      while ( $i \neq n$ ) {
7           $i := i + 1$ ;
8           $j := j + 1$ ;
9      }
10     return;
11 }
```

Q1. Does VerySimple satisfy the function specification?

Q2. Can we express lines 6–9 as a finite-length FOL formula?

Loop Invariant

```
1  @pre:  $n \geq 0$ 
2  @post:  $j = n$ 
3  void VerySimple (int  $n$ ) {
4      int  $i := 0$ ;
5      int  $j := 0$ ;
6      while ( $i \neq n$ ) {
7           $i := i + 1$ ;
8           $j := j + 1$ ;
9      }
10     return;
11 }
```

Q1. Does VerySimple satisfy the function specification?

Q2. Can we express lines 6–9 as a finite-length FOL formula?

We need to summarize the behaviors of loops. In our example, $i = j$ is the summarization that is precise enough to prove the partial correctness.

$$i = j \wedge i = n \rightarrow j = n$$

Loop Invariant

Each loop should be annotated with a proper loop invariant F .

```
1 while
2   @ $F$ 
3   ( $\langle condition \rangle$ ) {
4    $\langle body \rangle$ 
5 }
```

Loop invariant F is a property that (i) holds before the entrance and (ii) is preserved by executions of the loop body. That is, F holds at the beginning of every iteration. Thus,

- When entering the body, $F \wedge \langle condition \rangle$ holds.
- When exiting the loop, $F \wedge \neg \langle condition \rangle$ holds.

cf) A trivial loop invariant *true* holds for every loop, but it is useless in most cases.

Example 1: Linear Search

Find a nontrivial loop invariant to prove the correctness.

```
1  @pre:  $0 \leq l \wedge u < |a|$ 
2  @post:  $rv \leftrightarrow \exists i. l \leq i \leq u \wedge a[i] = e$ 
3  bool LinearSearch (int  $a[]$ , int  $l$ , int  $u$ , int  $e$ ) {
4      int  $i := l$ ;
5      while
6      @L:
7      ( $i \leq u$ ) {
8          if ( $a[i] = e$ ) return true;
9           $i := i + 1$ ;
10     }
11     return false;
12 }
```



Example 1: Linear Search

Find a nontrivial loop invariant to prove the correctness.

```
1  @pre:  $0 \leq l \wedge u < |a|$ 
2  @post:  $rv \leftrightarrow \exists i. l \leq i \leq u \wedge a[i] = e$ 
3  bool LinearSearch (int  $a[]$ , int  $l$ , int  $u$ , int  $e$ ) {
4      int  $i := l$ ;
5      while
6          @L:  $l \leq i \wedge (\forall j. l \leq j < i \rightarrow a[j] \neq e)$ 
7          ( $i \leq u$ ) {
8          if ( $a[i] = e$ ) return true;
9           $i := i + 1$ ;
10     }
11     return false;
12 }
```

- The index i is no less than l .
- We have not found an element in the previously examined indices j .

Example 2: Bubble Sort

```
1  @pre:  $\forall a. |a| \geq 0$ 
2  @post: sorted( $rv, 0, |rv| - 1$ )
3  bool BubbleSort (int  $a[]$ ) {
4    int[]  $a := a_0$ ;
5    @ $L_1$  :  $\left\{ \begin{array}{l} -1 \leq i < |a| \\ \wedge \text{partitioned}(a, 0, i, i + 1, |a| - 1) \\ \wedge \text{sorted}(a, i, |a| - 1) \end{array} \right\}$ 
6    for (int  $i := |a| - 1$ ;  $i > 0$ ;  $i := i - 1$ ) {
7      @ $L_2$  :  $\left\{ \begin{array}{l} 1 \leq i < |a| \wedge 0 \leq j \leq i \\ \wedge \text{partitioned}(a, 0, i, i + 1, |a| - 1) \\ \wedge \text{partitioned}(a, 0, j - 1, j, j) \\ \wedge \text{sorted}(a, i, |a| - 1) \end{array} \right\}$ 
8      for (int  $j := 0$ ;  $j < i$ ;  $j := j + 1$ ) {
9        if ( $a[j] > a[j + 1]$ ) {
10          int  $t := a[j]$ ;
11           $a[j] := a[j + 1]$ ;
12           $a[j + 1] := t$ ;
13        }
14      }
15    }
16    return  $a$ ;
17 }
```

$\text{partitioned}(a, l_1, u_1, l_2, u_2) \iff \forall i, j. l_1 \leq i \leq u_1 < l_2 \leq j \leq u_2 \rightarrow a[i] \leq a[j]$

Assertion

- The other formal comments on expected program behaviors.
- Usually specified by the command `assert` in most programming languages. If assertion violations occur at runtime, typically abort program executions; very useful for debugging errors.

```
1  @pre:  $0 \leq l \wedge u < |a|$ 
2  @post:  $rv \leftrightarrow \exists i. l \leq i \leq u \wedge a[i] = e$ 
3  bool LinearSearch (int a[], int l, int u, int e) {
4      int i := l;
5      while
6          @L :  $l \leq i \wedge (\forall j. l \leq j < i \rightarrow a[j] \neq e)$ 
7          ( $i \leq u$ ) {
8          @0  $0 \leq i < |a|$  /* expectation: array access is legal */
9              if (a[i] = e) return true;
10             i := i + 1;
11         }
12     return false;
13 }
```

Assertion

- Many kinds of assertions can be automatically generated to catch common semantic errors.
 - ▶ division-by-zero, null-dereference, accessing an array out of bounds, etc.
- For example, we can transform the C code snippet

$\dots; i := i/j; \dots$

into

$\dots; i := i/j; \text{assert}(j \neq 0); \dots$

Summary

- Goal: prove the correctness of implementations
- We explored specification methods to rigorously describe correct behavior using FOL formulas.
 - ① Function specification: precondition, postcondition
 - ② Loop invariant: summarization of loops
 - ③ Assertion: The other formal comments on expected behaviors

Next class: **Inductive assertion method** for proving partial correctness